Guide: Building Random Fields from the Compressor Blade Dataset for Geometry Perturbation

This guide explains how to convert the measured compressor-blade machining-deviation dataset into spatially correlated random fields that drive the geometry perturbation step (Stage 3) of the Robust Topology Optimization pipeline described in your masterContext. [1]

What the Data Actually Is

The xlsx contains 100 blade samples, each measured at 13 equidistant sections (H1 root to H13 tip), across 7 deviation types: leading-edge radius (ΔRLE), trailing-edge radius (ΔRTE), maximum thickness (ΔRMax), chord length (Δc), pressure-profile error (e_pp), suction-profile error (e_sp), and stagger angle (Δθ). Each sub-table is a 100×13 matrix — rows are blade samples, columns are blade-height sections — giving 9,100 measured values per type and 7 such matrices total. Critically, the paper notes these deviations are **not uniformly Gaussian**: several types show skewed or bimodal histograms and spatial trends along blade height (e.g., ΔRLE transitions from thinning at the root to thickening at the tip). This non-Gaussian, spatially-correlated structure is exactly why a Karhunen-Loève (KL) random field — not a simple scalar noise term — is required, matching the "Do NOT use scalar noise" rule in your masterContext. [2] [1]

Step-by-Step Method

Step 1 — Ingest and organize the data (`src/metrology/ingestion.py`)

- Load each of the 7 sub-tables into a `[100 samples × 13 sections]` array, with H1…H13 mapped to known blade-height (or arc-length) coordinates from the nominal CAD.
- Treat each deviation type as a **separate scalar random field** defined over the 1D blade-height coordinate (or, for profile errors e_pp/e_sp, over the 2D surface if richer spatial resolution is later available).
- This matches the masterContext's expected output: "cleaned points and deviations arrays" feeding into `registration.py/deviation.py`. [1]

Step 2 — Compute empirical statistics at each section (per Eqs. 1–4 in the paper)

For each section Hi and deviation type, compute:

- Mean (systematic deviation) μ(Hi) — Eq. 1 [2]
- Standard deviation (scatter) σ(Hi) — Eq. 2 [2]

- Skewness Sk(Hi) — Eq. 3, to flag non-Gaussian sections [2]
- Extreme relative deviation δ = x_extreme / y_design — Eq. 4, useful for sanity-checking tolerance bounds in Table 1 [2]

These per-section μ(x) and σ(x) become the **mean trend** and **marginal variance** of the random field Z(x), feeding the masterContext formula `Z(x) = μ(x) + Σᵢ √λᵢ φᵢ(x) ξᵢ`. [1]

Step 3 — Estimate the empirical spatial covariance (this IS the random field's DNA)

- With 100 samples × 13 sections, build the empirical 13×13 covariance matrix C(Hi, Hj) = cov(deviation at Hi, deviation at Hj) across the 100 blades, for each deviation type.
- This captures how an error at the root correlates with error at the tip on the *same physical blade* — the spatial correlation structure a scalar noise model would destroy.
- Fit a stationary kernel `k(x,x') = σ² exp(−‖x−x'‖²/2l²)` (squared-exponential or Matérn) to this empirical covariance via maximum likelihood or variogram fitting, exactly as specified in masterContext Section 3.3. Use OpenTURNS for this fitting step — never hand-code it. [1]

Step 4 — Karhunen-Loève decomposition (`OpenTURNS.KarhunenLoeveP1Algorithm`)

- Feed the fitted covariance kernel (or directly the empirical covariance matrix) into OpenTURNS' KL solver to obtain eigenpairs (λᵢ, φᵢ(x)).
- Truncate at order N_KL where cumulative eigenvalue sum captures ≥95% of total variance — this is a hard verification gate in masterContext. [1]
- Since several deviation types are non-Gaussian (skewed/bimodal per Table 3 in the PDF), use OpenTURNS to fit the marginal distribution of ξᵢ empirically (not assume standard normal) — e.g., via `ot.HistogramFactory` or `ot.KernelSmoothing` on the projected KL coefficients, then couple to a copula if cross-mode dependence exists. [2]

Step 5 — Validate the random field before use

- Resample synthetic fields from the fitted KL model and compare their sample covariance against the empirical covariance matrix from Step 3 — required verification gate. [1]
- Cross-check that synthetic realizations stay within the manufacturing tolerance bounds in Table 1 of the PDF (e.g., ΔRLE within ±0.13 mm), and that the reproduced histograms match the measured shapes (Gaussian, skewed, or bimodal) in Table 3. [2]

Step 6 — Map KL coefficients to mesh perturbation (`src/random_fields/perturbation.py` via Gmsh)

- Each of the 7 deviation types perturbs a different geometric feature of the blade surface mesh: ΔRLE/ΔRTE deform the leading/trailing-edge fillet radius, ΔRMax and Δc scale the thickness/chord profile, e_pp/e_sp displace pressure/suction surface nodes along the local normal, and Δθ rotates the section about the stacking axis.
- For a sampled ξ vector, reconstruct Z(x) at each blade-height section using the truncated KL sum, then apply that displacement to the corresponding mesh nodes in Gmsh, preserving positive Jacobians as required. [1]

Field Construction at a Glance

Quantity	Source step	Tool
$\mu(x)$, $\sigma(x)$ per section	Eqs. 1–2 on xlsx data [2]	NumPy/pandas
Empirical 13×13 covariance	Cross-section covariance over 100 samples [2]	NumPy
Fitted kernel $k(x,x')$	MLE/variogram fit to covariance	OpenTURNS
KL eigenpairs (λ_i, ϕ_i)	KarhunenLoeveP1Algorithm	OpenTURNS
Marginal of ξ_i	Histogram/KDE fit (non-Gaussian aware) [2]	OpenTURNS
Mesh deformation	Apply $Z(x)$ sample to nodes	Gmsh

Important Caveats From the Source Data

The dataset's authors explicitly warn that assuming Gaussian distributions for these deviations introduces "unpredictable quantitative error" in downstream UQ, so do not skip Step 2's skewness check before fitting your KL coefficient marginals. They also validated that 40 samples are sufficient for convergence of mean/variance statistics (shown for stagger angle at H13), which gives confidence that your 100-sample covariance estimate in Step 3 is statistically stable — but it is still wise to bootstrap the covariance matrix to estimate its own uncertainty before locking in kernel parameters for the robust optimization loop. [2]

⁂

1. masterContext.md
2. compressor_blade_deviations.pdf
3. Measured-machining-deviation-data-of-aero-engine-compressor-blades.xlsx